

PROGRAMME DE FORMATION · VERSION 3 · 2026-09-13

# Quarkus : bâtir une architecture microservices avec Quarkus

Développez un service Quarkus, de l'API REST à son exploitation. Le parcours aborde persistance, tests, échanges réactifs, comparaison JVM/native et déploiement sur un environnement Kubernetes d'exercice.

3 jours · 21 heures

Intermédiaire · Hybride

1 672 € HT

Tarif indicatif, dates, disponibilité et modalités contractuelles à confirmer avec le centre.

## À qui s'adresse cette formation ?

Développeurs Java et architectes souhaitant construire et exploiter des services avec Quarkus.

### Prérequis

Pour suivre cette formation Quarkus, il est nécessaire de savoir développer en Java, de posséder une connaissance des architectures Web, une connaissance du protocole HTTP et des concepts REST, ainsi que d'avoir des notions de déploiement de conteneurs.

## Objectifs pédagogiques

- Mettre en place un IDE et les outils de développement pour quarkus
- Développer différents types de microservices avec Quarkus (API RestFul, Messagerie réactive, Client Rest)
- Comprendre la compilation AOT et les phases de build d'une application Quarkus
- Déployer et surveiller une architecture micro-services Quarkus vers Kubernetes

## Le cas fil rouge

Développer et exploiter deux microservices Quarkus échangeant des commandes par API et messagerie.

### Préparation

Java, HTTP et conteneurs ; JDK, Maven, Quarkus et cluster Kubernetes de démonstration préparés.

## Progression détaillée

Répartition pédagogique proposée sur une base de 7 heures par jour, pauses exclues. À ajuster avec le formateur au positionnement initial. Les approfondissements « Pour aller plus loin » sont sélectionnés par le formateur selon le positionnement et le temps disponible ; ils ne constituent pas des heures supplémentaires garanties.

## Jour 1 · 7 heures

### 1. Préparer le projet Quarkus

210 minutes · apports, exercice et retour

- Comprendre extensions, injection et configuration par profil.
- Utiliser le mode développement et les tests continus.

- Définir frontières des services et contrats d'échange.
- Structurer configuration, injection de dépendances et profils ; isoler les secrets.

#### NOTIONS ET MÉTHODES EN DÉTAIL

- Créer un projet avec extensions nécessaires et configuration par profil.
- Utiliser injection de dépendances et mode développement en distinguant commodité locale et exploitation.
- Définir structure du code, dépendances et configuration externe du service.

#### Atelier prévu

Créer un service commandes et isoler sa configuration locale, de test et de déploiement.

#### Livrable attendu

Projet Quarkus configuré et reproductible.

#### Vérification en atelier

Le projet démarre avec un profil documenté sans secret commité.

Pour aller plus loin · selon positionnement et temps disponible

Détecter une différence de comportement entre profil de test et production.

## 2. Exposer et consommer des API REST

210 minutes · apports, exercice et retour

- Définir ressources, validation et réponses d'erreur.
- Appeler un autre service avec un client REST typé.
- Encadrer délais, erreurs distantes et autorisations.
- Définir contrats REST, validation, codes d'erreur, timeouts et compatibilité des API.

#### NOTIONS ET MÉTHODES EN DÉTAIL

- Définir ressources REST, validation des entrées et représentation homogène des erreurs.
- Consommer une API avec timeout et traitement des statuts non réussis.
- Séparer DTO, logique métier et accès externe pour faciliter les tests.

#### Atelier prévu

Relier commandes et catalogue puis simuler un catalogue indisponible pour vérifier la réponse de secours.

#### Livrable attendu

API documentée et tests de contrat.

#### Vérification en atelier

Une API distante indisponible produit une erreur maîtrisée et une réponse utile.

Pour aller plus loin · selon positionnement et temps disponible

Simuler une dépendance lente et vérifier les erreurs renvoyées au client.

## Jour 2 · 7 heures

### 3. Persister et tester les données

210 minutes · apports, exercice et retour

- Organiser entités, accès aux données et transactions.

- Gérer migrations et contraintes métier.
- Tester les ressources et les dépendances avec services de laboratoire.
- Maîtriser transactions, concurrence, migrations et tests d'intégration de persistance.

#### NOTIONS ET MÉTHODES EN DÉTAIL

- Modéliser entités, transactions et requêtes ; repérer chargement excessif et problème N+1.
- Comparer test unitaire, test Quarkus et test avec base de démonstration.
- Préparer contraintes et données de test isolées pour les scénarios d'échec.

#### Atelier prévu

Enregistrer une commande de façon atomique et démontrer le rollback lorsque la validation d'une ligne échoue.

#### Livrable attendu

Couche de persistance et tests transactionnels.

#### Vérification en atelier

Une erreur transactionnelle ne laisse pas de modification partielle.

Pour aller plus loin · selon positionnement et temps disponible

Reproduire une mise à jour concurrente et choisir une stratégie de verrouillage.

## 4. Introduire les échanges réactifs

210 minutes · apports, exercice et retour

- Comprendre messages, canaux, accusés de réception et reprises.
- Éviter les traitements bloquants dans un chemin réactif.
- Gérer doublons et erreurs de consommation.
- Distinguer traitement bloquant et non bloquant ; traiter pression, erreurs et retries.

#### NOTIONS ET MÉTHODES EN DÉTAIL

- Distinguer appel bloquant, traitement asynchrone et flux réactif selon la ressource utilisée.
- Examiner backpressure, reprise et propagation d'erreur sur un petit flux de messages.
- Définir gestion des doublons et frontière de transaction lors d'une consommation.

#### Atelier prévu

Publier un événement commande puis traiter deux fois le même message sans créer deux expéditions.

#### Livrable attendu

Flux réactif et stratégie d'idempotence.

#### Vérification en atelier

Un message répété n'entraîne pas deux effets métier non contrôlés.

Pour aller plus loin · selon positionnement et temps disponible

Éviter une duplication métier après reprise d'un message.

## Jour 3 · 7 heures

### 5. Comparer build JVM et compilation native

210 minutes · apports, exercice et retour

- Expliquer phases de build et contraintes de compilation AOT.
- Mesurer démarrage, mémoire et temps de construction.
- Identifier ressources et bibliothèques nécessitant une configuration spécifique.
- Comparer démarrage, mémoire, compilation et contraintes de réflexion entre JVM et natif.

#### NOTIONS ET MÉTHODES EN DÉTAIL

- Mesurer démarrage, mémoire et temps de réponse avec le même scénario JVM et natif.
- Repérer réflexion, ressources et bibliothèques demandant une configuration pour le natif.
- Comparer coût de construction et contraintes d'exploitation avant de choisir.

#### Atelier prévu

Comparer deux artefacts du même service et documenter les limites observées de la version native.

#### Livrable attendu

Rapport de mesures et choix d'exécution.

#### Vérification en atelier

Le choix s'appuie sur mesures et compatibilités du service étudié.

#### Pour aller plus loin · selon positionnement et temps disponible

Mesurer deux variantes avec le même scénario sans généraliser un microbenchmark.

## 6. Déployer et surveiller sur Kubernetes

210 minutes · apports, exercice et retour

- Configurer déploiement, service, secrets et ressources.
- Distinguer disponibilité, démarrage et santé fonctionnelle.
- Collecter métriques et traces pour diagnostiquer une panne.
- Séparer readiness et liveness ; configurer ressources, traces et déploiement progressif.

#### NOTIONS ET MÉTHODES EN DÉTAIL

- Définir image, configuration, ressources et probes de disponibilité ou de santé.
- Séparer logs, métriques et traces ; relier un incident à une requête de test.
- Préparer déploiement progressif et retour à une image précédente.

#### Atelier prévu

Démontrer le déploiement de laboratoire et une reprise après arrêt d'un pod ; remettre contrats, tests et guide d'exploitation.

#### Livrable attendu

Manifestes et guide de diagnostic.

#### Vérification en atelier

Le service ne reçoit du trafic qu'une fois prêt et son retour arrière est vérifié.

#### Pour aller plus loin · selon positionnement et temps disponible

Diagnostiquer un service sain localement mais indisponible derrière le cluster.

## Évaluation finale

Mise en situation individuelle et restitution commentée, incluses dans la dernière séquence. Durée indicative : 60 minutes.

- Les appels distants et messages gèrent les erreurs attendues.
- La comparaison JVM/native repose sur des mesures.
- Les contrôles de santé et les paramètres de déploiement sont documentés.

## Méthodes et ressources

Apports illustrés, exercices progressifs, ateliers sur un cas fil rouge et retour argumenté sur les productions. Les ateliers et l'évaluation font partie des heures indiquées.

Les ateliers décrivent les travaux prévus. Leurs fichiers, corrigés et évaluations en ligne sont à préparer, sauf ressources explicitement disponibles dans le module pilote.

## Accès, accompagnement et contact

Pour obtenir plus de détails ou le programme détaillé validé, contactez le centre : 09 54 70 23 80 — [contact@beyondexpertise.eu](mailto:contact@beyondexpertise.eu).

Positionnement et vérification des prérequis avant inscription. Délais, calendrier et aménagements à convenir avec le centre. Pour un besoin d'accessibilité, décrivez le besoin pratique sans diagnostic médical.

Programme enrichi proposé, à valider par le formateur avant animation. Les approfondissements sont adaptés au niveau et au temps disponible. Aucune certification professionnelle ni prise en charge CPF ou OPCO n'est garantie. Démarche Qualiopi en cours ; certification non acquise.

## Références pour approfondir

- [Quarkus](#) · guides officiels